

软件工程

敏捷软件开发方法

刘 爽

中国人民大学

本课程部分教学内容基于：《软件工程-理论与实践》，毛新军、董威编著，高等教育出版社，2024.1，ISBN 978-7-04-061010-9 配套课件由出版社/作者提供，仅用于教学

内容

1. 何为敏捷开发方法

- ✓ 基本思想和原则
- ✓ 特点和应用

2. 具体敏捷开发方法

- ✓ 极限编程
- ✓ 测试驱动开发方法
- ✓ Scrum方法
- ✓ DevOps方法



1.1 传统重型软件开发方法的特点和不足

□遵循严格的过程和计划

- ✓ 定义广泛适用的过程并通过团队来执行该过程，从而指导软件开发
- ✓ 以预测性为主，倾向于预先制定详细的计划，并通过该计划来指导软件项目的实施，并期望软件开发过程与计划之间的偏差越少越好

□以文档为中心

- ✓ 通过文档记录各个阶段的成果，将文档作为交互的媒介
- ✓ 需对文档进行持续改进和评审
- ✓ 难以有效应对软件需求的变化

□等到开发后期才能得到可运行软件

- ✓ 编码是开发的后期工作

1.2 什么是敏捷方法 (Agile Method) ?

- 一种**轻量级**软件开发方法

- ✓ 相对于重量级的软件开发方法而言

- 主张软件开发要以**代码为中心**，**快速、轻巧和主动应对需求变化**，**持续、及时交付可运行的软件系统**

- ✓ 轻便、轻巧

- 提供了一组**思想和策略**，**指导快速响应用户需求的变化**，**快速交付可运行的软件制品**

1.3 敏捷开发方法的基本价值观

- 较之于过程和工具，应更加重视**人和交互**的价值
- 较之于面面俱到文档，应更加重视**可运行软件系统**的价值
- 较之于合同谈判，应更加重视**客户合作**的价值
- 较之于遵循计划，应更加重视**响应用户需求变化**的价值

要以快速满足用户需求为目标，以此来改变软件开发的理念、思想和方法

敏捷方法的指导思想

□强化可运行的软件，弱化软件文档

- ✓以可运行软件为中心来开展软件开发

□以适应变化为目的，提升应变能力

- ✓针对变化不断进行优化和调整任务、产品和计划等

□以人为本，让方法等服务于人

- ✓敏捷软件开发是面向人的而不是面向过程的，让方法、技术、工具、过程等来适应人，而不是让人来适应它们

敏捷意味着：轻盈、灵巧；无过多的负担；迅速响应变化

1.4 敏捷准则

□基于四项价值观，人们进一步提出12条敏捷开发原则

✓以指导开发人员运用敏捷开发方法来开发软件实践

□这些原则使得敏捷方法更具可操作性



敏捷准则(1/2)

1. **尽早和持续地交付**有价值的软件，以使用户满意
2. 即使到了软件开发后期，也**欢迎用户需求的变化**
3. **不断交付**可运行软件系统，**交付周期**可以从几周到几个月
4. 在整个软件项目开发期间，用户和开发人员最好能每天**一起工作**
5. 由**积极主动**的人来承担项目开发，给他们提供所需环境和支持，信任他们的能力
6. 团队内部最有效的信息传递方式是**面对面的交谈**

敏捷准则(2/2)

- 7. 将**可运行软件**作为衡量软件开发进度的首要标准
- 8. 可持续性的开发，出资方、开发方和用户方应当保持**长期、恒定的开发速度**
- 9. 关注**优秀的技能和良好的设计**会增强敏捷性
- 10. 采用**简单化**的方法来完成任任务
- 11. 最好的架构、需求和设计出自于**自组织的团队**
- 12. 软件开发团队应定期就如何提高工作效率的问题进行**反思**，并进行相应的**调整**

敏捷软件开发对技术提出的要求

- 如何快速开发出可运行的软件系统？
- 当需求改变时，如何快速应对变化？
- 如何给出可有效应对变化的软件设计？
- 在文档缺乏情况下如何保证软件质量？
- 如何提高软件开发的效率？

1.5 支持敏捷软件开发的技术

- ❑ 极限编程(eXtreme Programming,XP)
- ❑ 测试驱动开发 (Test Driven Development,TDD)
- ❑ Scrum方法
- ❑ DevOps方法
- ❑ 适应性软件开发 (Adaptive Software Development, ASD)
- ❑ 特征驱动开发 (Feature Driven Development,FDD)
- ❑ 敏捷设计
- ❑ 动态系统开发方法 (Dynamic System Development Method,DSDM)

敏捷开发方法是一大类方法统称，它们均遵循敏捷思想

内容

1. 何为敏捷开发方法

- ✓ 基本思想和原则
- ✓ 特点和应用

2. 具体敏捷开发方法

- ✓ 极限编程
- ✓ 测试驱动开发方法
- ✓ Scrum方法
- ✓ DevOps方法



2.1 极限编程的基本思想

□由Kent Beck提出的一种特殊的敏捷软件开发方法

□四条核心思想

- ✓**交流**，强调基于口头（而非文档、报表和计划）的交流
- ✓**反馈**，通过持续、明确反馈来获得软件状态
- ✓**简单**，用最简单的技术来解决问题
- ✓**勇气**，快速开发并在必要时具有重新进行开发的信心

□将经过数十年检验的准则结合在一起，定义了**五条**指导性原则和**十二条**须遵循的核心准则

极限编程的5条指导原则

□ 快速反馈

- ✓ 从用户处迅速得到有关软件的反馈，确认开发是否满足用户需求，通过自动化测试迅速了解软件运行状况

□ 简单性假设

- ✓ 开发人员只考虑当前迭代所面临问题，无需考虑下一次迭代的问题，用简单方法和技术来解决问题。

□ 逐步更改

- ✓ 通过一系列修改来逐步解决问题和完善系统，不要期望一次迭代就开发出完整的软件系统。

□ 支持变化

- ✓ 欢迎用户改变需求，支持用户需求动态变化。

□ 高质量的工作

- ✓ 采用诸如测试驱动开发等技术高质量地开展工作，确保软件质量。

极限编程的12条核心准则(1/3)

□计划游戏

- ✓ 软件开发团队快速制定下一次迭代的软件开发计划

□隐喻(Metaphor)

- ✓ 使用业务相关术语来描述需求，促使开发人员和业务人员对系统达成共同和一致的理解

□小型发布

- ✓ 经常性发布可运行软件系统，每次发布的软件系统仅提供少量功能

□简单设计

- ✓ 只为当前的需求做设计，程序能运行所有测试、没有重复逻辑、包含尽可能少的类和方法

极限编程的12条核心准则(2/3)

□测试

- ✓测试应在编写代码之前进行

□重构

- ✓在不改变程序代码功能的前提下，改进程序代码的设计，使程序代码更加简单，更易于扩展

□结对编程

- ✓两名程序员同时在一台计算机上共同开展编程工作

□代码集体拥有

- ✓开发小组的任何成员都可以查看并修改任何部分的代码

极限编程的12条核心准则(3/3)

□持续集成

- ✓经常性地集成

□每周工作40小时

- ✓倡导质量优先

□现场用户

- ✓用户代表在现场办公，参与开发全过程，确保能及时得到反馈

□编码标准

- ✓遵循统一编码标准，以提高软件系统的可理解性和可维护性

2.2 传统软件开发的局限

□程序员先编写程序代码，然后再对程序代码进行测试

- ✓基于已有的程序代码进行测试

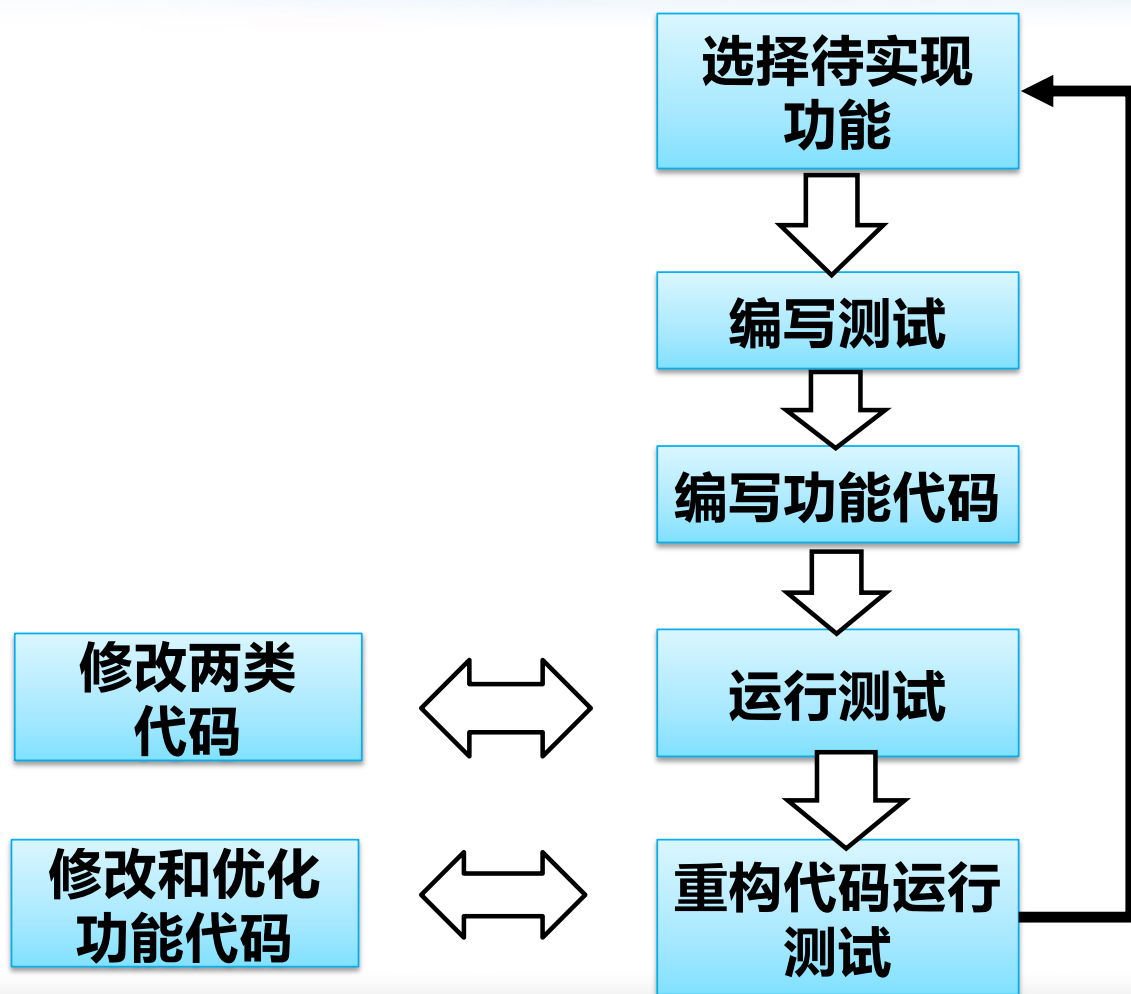
□局限性

- ✓**测试常被视为是附加工作**，由于进度压力，常被忽视，导致没有足够时间对代码进行详尽和充分的测试
- ✓当文档与程序代码不一致时，对程序代码进行测试就会存在许多问题
- ✓测试通常是在程序代码编写完成之后才进行的，因而**无法保证编写程序和测试同步**
- ✓测试被视为是乏味的工作，人员缺乏积极性和成就感

测试驱动开发的思想

- **在编写业务代码之前，先确定和编写测试**
- **程序员首先要思考如何对某个功能进行测试，设计好相应的测试用例，编写好相关的测试代码，然后编写相应的程序代码以通过软件测试**

测试驱动开发的过程



测试驱动开发的特点

- 根据**测试**来编写代码
- 编写测试的目的不仅是为了测试程序代码能够正常工作，而且被用于定义程序代码的内涵
- 确保任何程序代码都是**可测试**的
- 编码完成后即完工
- 易于维护 、 质量保证

2.3 Scrum方法

□旨在通过**增量或迭代**的方式加强软件项目的管理以及**快速提交**可运行的软件产品



Scrum方法的大致流程

□首先，产品所有者需创建**软件产品订单库**即 “Backblog”

- ✓ 描述软件产品需提供的功能需求以及它们的优先级排序

□其次，筛选出**最应该实现的**软件需求，

- ✓ Scrum主人基于 “Backblog” 中各项软件需求及其优先级，形成待实现的软件产品冲刺订单库，即 “SprintLog”

□然后，软件开发将进入冲刺 “**Sprint**” 周期

- ✓ 以实现选定软件订单，每个冲刺就是一次增量开发，一般持续1到4周

□最后，共同开展**Scrum评审**

- ✓ 一次冲刺完成后，每个团队成员演示自己的开发成果，大家共同审查成果是否高质量地实现了既定功能，并就其中的问题进行反思，以指导和改进下一次冲刺

2.4 DevOps方法的基本思想

□传统方法将**软件开发和运维**的分离开来

□DevOps是由 **“Development”** 和 **“Operations”** 二个英文单词的缩写连接而成

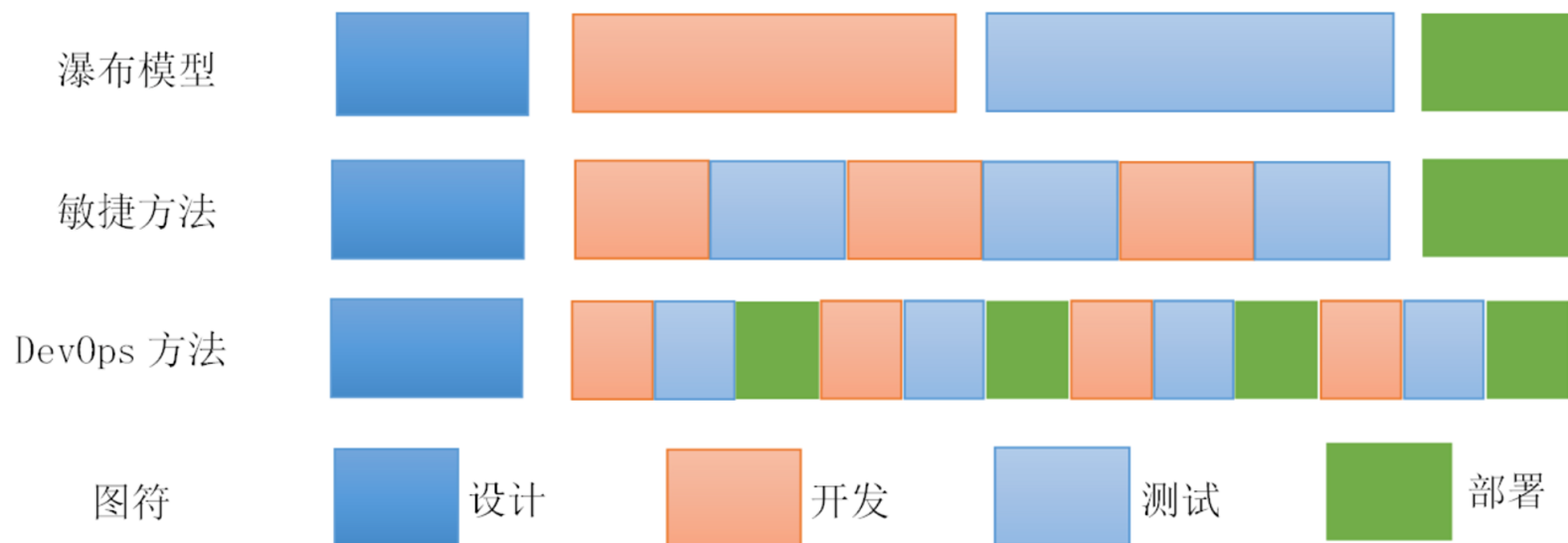
✓反映了要将软件开发和软件运维二个阶段的工作统一和集成在一起加以考虑，实现二者之间的无缝连接

□DevOps方法

✓将敏捷的理念和思想从**软件开发阶段**延伸到**软件运维阶段**

✓DevOps方法实际上是一种特殊的敏捷开发方法

DevOps方法与其他方法的对比



持续集成、持续交付和持续部署(1/2)

□持续集成 (Continuous Integration, CI)

- ✓软件开发团队成员经常性地集成他们的工作成果（如程序代码），把团队个人研发的部分代码向团队软件整体部分交付

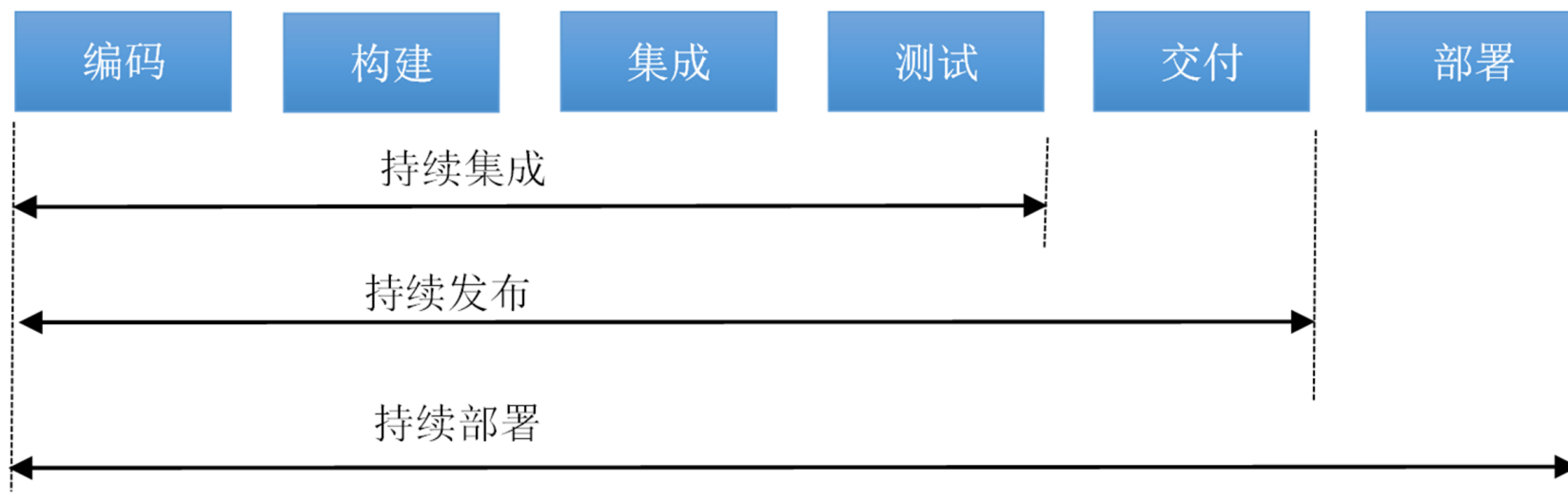
□持续交付 (Continuous Delivery, CD)

- ✓在持续集成的基础上，通过自动化测试，将集成后的代码部署到更贴近真实运行的环境之中，从而为持续部署奠定基础

□持续部署 (Continuous Deployment, CD)

- ✓当交付的代码通过评审之后，自动部署到软件系统的实际和真实运行环境中

持续集成、持续交付和持续部署(2/2)



DevOps的工具支持

□为了实现持续集成、持续交付和持续部署，DevOps方法需要借助于一系列的工具

- ✓常见的**持续集成CASE工具**包括Jenkins、GitLab CI、Travis CI、CircleCI、Maven等,它们可以与 GitLab、GitHub、Bitbucket等版本控制系统集成使用
- ✓常见的**持续交付和持续部署工具**包括Ansible、Puppet、Chef等，它们可以自动化部署和管理基础设施以及应用程序
- ✓使用**监控工具**（如 Nagios、Zabbix、Prometheus 等）对部署的应用程序进行监控和故障排查，对系统资源、应用程序性能、日志等方面进行监控，并提供报警、自愈等功能。

2.5 敏捷方法的特点

□小和少

- ✓生成少量软件文档，每个文档规模要小
- ✓每次迭代要实现软件功能的数量和规模要小，迭代周期要小

□简

- ✓技术、工具以及每次迭代要解决的问题尽可能简单
- ✓只关注当前欲实现的功能需求，而不要考虑将来的问题

□快

- ✓快速响应变化、从用户处获得反馈，给用户提交有价值软件，对软件产品进行迭代和更新

□变

- ✓允许需求动态变化，要以变应变，开发团队应是自组织的

重型方法与敏捷方法对比

方法名称	重型开发方法	敏捷开发方法
基本理念	以文档为中心	以代码为中心
交付特点	交付慢、开发持续时间长、难以快速应对需求变化	交付快、开发持续时间短、可快速应对需求的变化
适合项目	对软件文档和过程要求高、软件需求演变不快的软件系统，如要求遵循CMM规范的军用软件系统开发	需求经常变化的软件，如互联网软件
典型代表	瀑布模型、增量模型、迭代模型、原型模型、螺旋模型等	测试驱动开发、Scrum、DevOps方法等

示例：12306软件开发采用敏捷方法

- Mini-12306软件的边界不确定，需求动态变化，客户要求开发方尽早和持续交付软件系统**
- 针对该项目软件需求特点及交付要求，项目组可考虑采用迭代、敏捷的方法来开展该软件系统的开发**
- 每次迭代或每个Scrum周期针对若干明确的软件需求项开展开发工作，每隔一段时间给客户交付可运行软件系统**

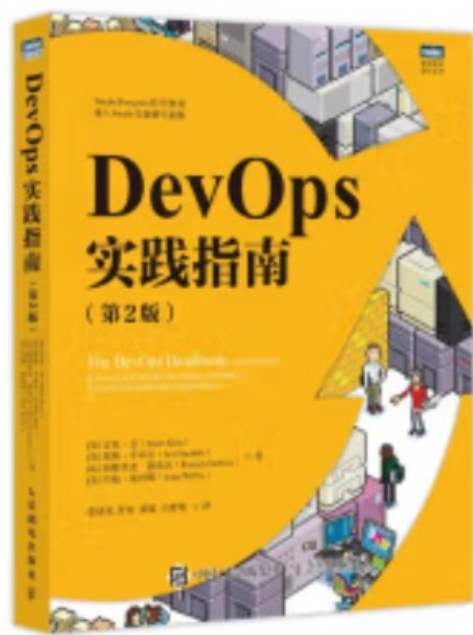
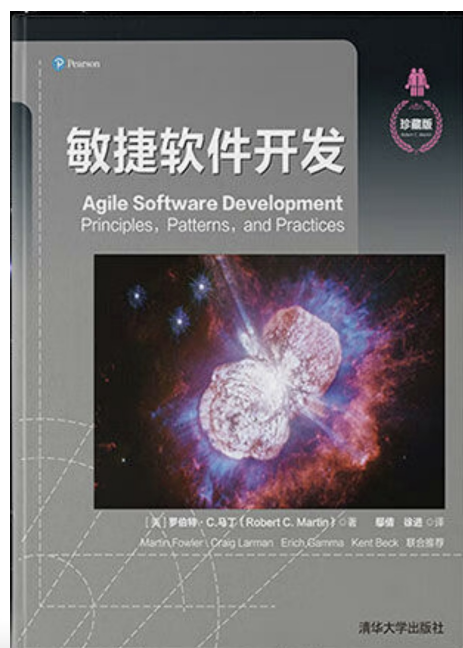
思考和讨论

- 敏捷开发方法与瀑布过程模型等的本质区别是什么？
- 敏捷开发方法适合于哪些类别的软件开发？
- DevOps方法如何体现敏捷开发的思想？

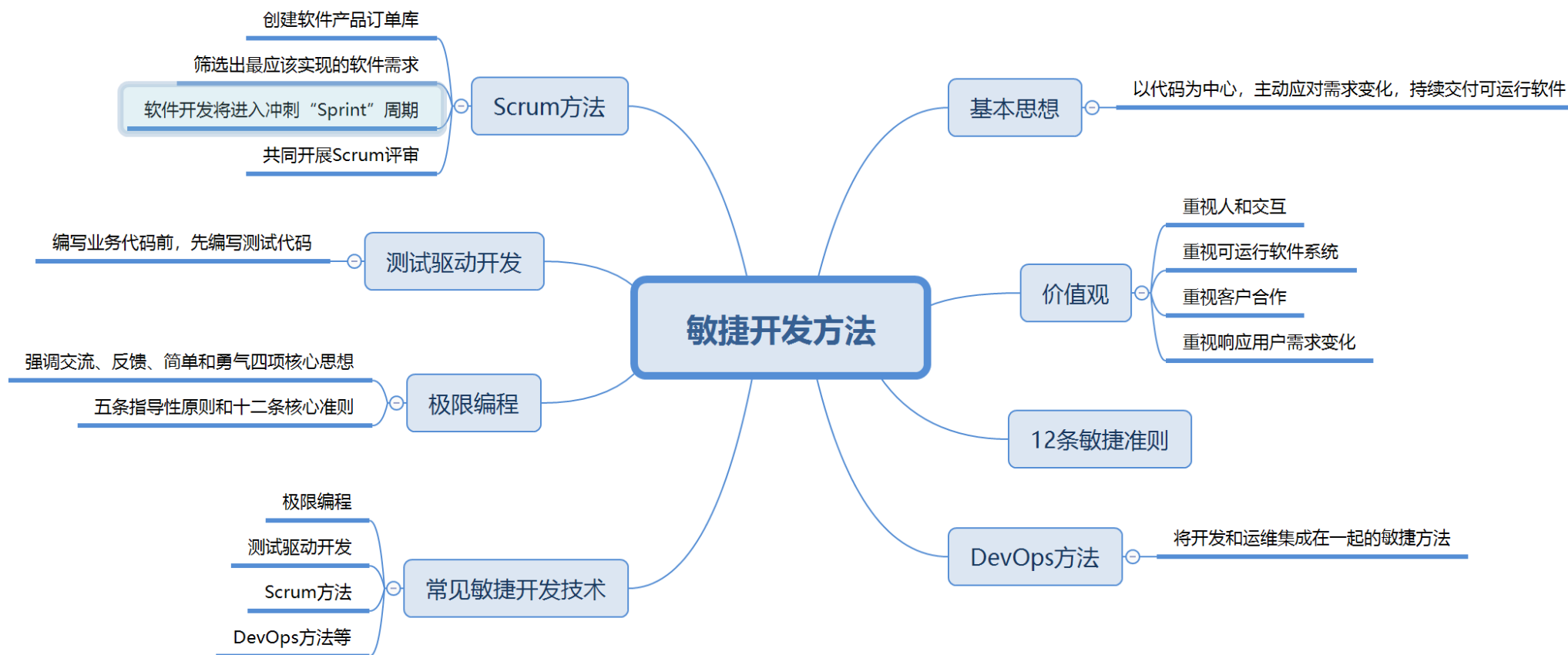


拓展阅读

- **敏捷软件开发**, [美] 罗伯特·C.马丁 (Robert, C., Martin), 清华大学出版社, 2020.
- **DevOps实践指南 (第2版)**, 人民邮电出版社, 2024.



本章知识图谱



小结

□敏捷开发方法的本质

- ✓应对软件**需求变化**，解决传统过程模型的不足

□敏捷开发方法的特点

- ✓小、简、快、变、体，**轻量级方法**，以**代码为中心**的方法

□敏捷开发方法的构成

- ✓由许多具体的方法组成，如**Scrum方法**、**极限编程**等